

Unit 2

AI Terminologies and Basic Concepts

1.1 Agents, Environment, and Actions

In Artificial Intelligence (AI), the concepts of *agents*, *environment*, and *actions* form the backbone of problem-solving. An **agent** is an entity that can perceive its environment through sensors and act upon that environment through actuators. The environment is everything that surrounds the agent and influences its behaviour. Actions are the moves or operations that an agent performs to achieve its goals.

Environment: Everything outside the agent that the agent interacts with is called the environment.

The interaction between an agent and its environment can be described as a **perception-action cycle**: the agent observes, processes, decides, and acts, then receives feedback from the environment.

AI developers design intelligent systems that can operate autonomously, adapt to changes, and optimize their performance.

Food delivery apps like **Zomato/Swiggy** use agents. The “agent” is the app’s AI, the “environment” is customer location, restaurant location, traffic conditions, and weather. The “actions” are assigning the best delivery partner, suggesting food options, or updating delivery time.

Real-life Example

- Self-driving car:
 - o Sensors: Camera, Radar, GPS
 - o Actuators: Steering, Brakes
 - o Environment: Road, traffic, pedestrians

Agents are classified into different types:

1. Simple Reflex Agents

Simple reflex agents act only based on the **current perception** of the environment, without considering history or future consequences. They use **condition–action rules** like “IF condition THEN action.” For example, a thermostat senses room temperature and turns the heater on or off depending on the threshold. These agents are fast and efficient in predictable environments but fail when the situation requires memory or learning.

2. Model-Based Agents

Model-based agents maintain an **internal model of the environment**. They don't just rely on current inputs but also consider **past information**. This helps them deal with partially observable environments. For example, a self-driving car uses sensors (camera, radar) plus memory of road rules to navigate safely. The model updates continuously as new perceptions arrive.

3. Goal-Based Agents

Goal-based agents make decisions based on achieving specific **goals**. They don't just react; they plan actions to reach a target state. For example, a chess-playing agent evaluates possible moves to checkmate the opponent. These agents use **search and planning algorithms** to find a path from the initial state to the goal state.

4. Utility-Based Agents

Utility-based agents go beyond goals; they evaluate which action maximizes **overall utility or happiness**. They use **utility functions** to compare options and choose the best trade-off. For example, a self-driving car may choose a slightly longer route if it avoids heavy traffic, maximizing comfort and efficiency. These agents are ideal for complex, dynamic environments.

State, Initial State, and Goal State

In Artificial Intelligence (AI), the idea of solving problems is often explained in terms of **states**. A **state** represents a particular situation or condition of the system at a given moment. It is like a sensor that captures all the necessary details of the environment. To solve any problem, an AI agent must know three things: the **initial state**, the **goal state**, and the **possible actions** to move between them.

1. State

A **state** is a description of the system's current status. It defines where the agent is in the problem-solving process. States can be physical (e.g., a robot's location) or abstract (e.g., arrangement of chess pieces on a board). By defining states, we can break down complex problems into manageable steps.

2. Initial State

The **initial state** is the starting point of the agent before it begins solving the problem. It is the first condition that defines the problem. For example, in Google Maps navigation, the initial state is the user's current location. In a chess game, the initial state is the standard board setup.

Clearly defining the initial state is essential because it provides the agent with a reference point to begin its search for solutions.

3. Goal State

The **goal state** represents the condition that the agent wants to achieve. It is the desired final outcome of the problem-solving process. For instance, in Google Maps, the goal state is the destination location. In an 8-puzzle problem, the goal state is the correctly arranged puzzle.

A goal state can be **specific** (e.g., reaching a city) or **satisfy a condition** (e.g., "any path shorter than 10 minutes"). AI agents often use **goal tests** to check whether the current state matches the goal.

Importance in Problem Solving

AI problem-solving can be summarized as:

- Start from the **initial state**.
- Use **actions** to move between states.
- Continue until the **goal state** is reached.

This process can be represented as a **state space**, where nodes are states and edges are actions.

Industry Example

In **E-commerce (Amazon/Flipkart)**:

- **Initial State**: Customer browsing products.
- **Action States**: Adding items to cart, applying discount, choosing delivery option.
- **Goal State**: Successful order placement.

Here, AI helps optimize actions (e.g., suggesting payment options, fastest delivery route) to move from initial state to goal state smoothly.

1.3 State Space Representation

In Artificial Intelligence (AI), **state space representation** is a method of describing a problem in terms of **states** and **actions**. A **state** is a snapshot of the system at a given time, and **actions** are the operations that move the system from one state to another. Together, all possible states and actions form the **state space** of the problem.

- A **state** is a specific condition or situation of the problem.
- **State space** is the complete set of all possible states that can exist.
- A solution is a sequence of actions that leads from the **initial state** to the **goal state**.

State space representation is the method of modeling a problem in terms of states and actions. It is a graph-like structure where:

- **Nodes** represent states (initial, intermediate, or goal).
- **Edges** represent actions leading from one state to another.

This representation allows AI systems to systematically search for solutions. The agent explores the state space until it finds a path from the initial state to the goal state.

The efficiency of problem-solving depends on how the state space is represented. A poorly defined state space can make the search unnecessarily large and computationally expensive, while a well-defined one simplifies problem-solving.

Classic examples include:

- **Route finding** (Google Maps uses states as locations and actions as roads).
- **Game playing** (Chess positions as states and moves as actions).

Important terms:

- **Initial State** – where the agent starts.
- **Goal State** – desired condition.
- **Actions/Operators** – moves that change one state into another.
- **Path Cost** – cost of the sequence of actions.

Example

In **logistics and supply chain**, state space representation is applied. Suppose Flipkart wants to deliver a package. The **states** are different warehouses and customer addresses. The **actions** are transport routes like trucks, flights, or bikes. AI uses the state space to find the shortest and cheapest delivery path.

1.4 Problem Formulation

In Artificial Intelligence (AI), **problem formulation** is the process of precisely defining a problem so that it can be solved systematically by an intelligent agent. If a problem is not formulated correctly, then AI system may waste time exploring irrelevant possibilities or fail solution.

A well-formulated problem provides the **structure** for the agent to move from the **initial state** to the **goal state** using appropriate actions.

In problem formulation the agent is to **focus only on relevant states** instead of exploring the entire environment. It provides a **framework for applying search strategies** (like BFS, DFS, A*).

Without correct formulation, the AI might attempt meaningless actions.

Key Components of Problem Formulation

1. **Initial State**
 - o The condition or situation where the agent starts.
 - o Example: In navigation, the initial state is the current location.
2. **Actions**
 - o The set of operations the agent can perform.
 - o Example: In chess, actions are legal moves of pieces.
3. **Transition Model**
 - o A description of how each action changes the state.
 - o Example: In the 8-puzzle, sliding a tile changes the board configuration.
4. **Goal Test**
 - o A check to determine if the current state is the goal.
 - o Example: In a delivery system, goal test = “Is the package delivered to the customer?”
5. **Path Cost**
 - o A numerical value representing the cost of reaching a state. It helps the agent choose the most efficient path.
 - o Example: In logistics, cost may be calculated as time, distance, or fuel.

For example, if Uber doesn’t correctly define pickup and drop locations, the ride cannot be completed, no matter how advanced the system is.

Industry Example

In Uber or Ola ride-booking:

- **Initial State:** Rider’s current location.
- **Actions:** Assigning drivers, selecting routes, calculating fares.
- **Transition Model:** Choosing a specific driver and route changes the state (driver moves toward pickup).
- **Goal Test:** Rider successfully dropped at the destination.
- **Path Cost:** Travel time, surge pricing, or distance covered.

Search Strategies

In Artificial Intelligence (AI), search strategies are systematic methods used by an agent to explore the **state space** and find a path from the **initial state** to the **goal state**. Since many AI problems can be framed as search problems (e.g., route finding, puzzle solving, scheduling), choosing the right search strategy is essential for efficiency.

A **search strategy** defines the order in which states are explored, the rules for expanding nodes, and how the algorithm decides the next step.

Broadly, search strategies are of two types:

Uninformed (Blind) Search

In Artificial Intelligence (AI), **Uninformed Search** is a type of search strategy where the agent has **no extra knowledge** about the problem domain except the basic definition: **initial state, possible actions, and goal test**. The term “blind” means the algorithm does not know which path is better; it explores the state space systematically but without guidance.

These algorithms are fundamental because they guarantee a solution if one exists, but they can be slow and memory-heavy for large problems.

Key Characteristics

1. **No domain-specific knowledge** → Only uses problem definition.
2. **Explores blindly** → Checks nodes systematically without preference.
3. **Completeness** → Guaranteed to find a solution if one exists.
4. **Optimality** → Some are optimal (e.g., Breadth-First Search), some are not (e.g., Depth-First Search).
5. **Informed (Heuristic) Search** – Uses extra knowledge (heuristics) to guide the search efficiently.

Main Types of Uninformed Search

1. **Breadth-First Search (BFS)**
 - Explores nodes level by level.
 - Always finds the shortest path (if all actions have the same cost).
 - Example: In social networks, finding the shortest connection (friend-of-a-friend).
2. **Depth-First Search (DFS)**
 - Explores as deep as possible before backtracking.
 - Memory efficient but may get stuck in infinite paths.
 - Example: Exploring folder structures in a computer.
4. **Depth-Limited Search (DLS)**
 - Like DFS but with a predefined depth limit.
 - Prevents infinite loops but may miss solutions beyond the limit.
5. **Iterative Deepening Search (IDS)**
 - Combines BFS and DFS: increases depth limit step by step.
 - Saves memory like DFS but guarantees optimality like BFS.

Informed (Heuristic) Search

In Artificial Intelligence (AI), **Informed Search** is a search strategy that uses **extra knowledge (heuristics)** about the problem domain to guide the search toward the goal more efficiently. Unlike **Uninformed Search**, which explores blindly, informed search makes “educated guesses” about which path is likely to lead to the goal faster.

The central idea is the **heuristic function ($h(n)$)**, which estimates the cost of reaching the goal from a given state. The quality of the heuristic directly affects the efficiency of the search.

Key Features

1. **Uses heuristic knowledge** → Guides the search intelligently.
2. **Efficient** → Explores fewer states compared to uninformed search.
3. **Goal-directed** → Focuses on promising paths first.
4. **Depends on heuristic quality** → A good heuristic = faster solution, a poor heuristic = nearly blind search.

Popular Informed Search Algorithms

1. Greedy Best-First Search

- Chooses the path that appears closest to the goal (minimum $h(n)$).
- Fast but not always optimal.
- Example: In Google Maps, choosing the route that looks shortest on the map.

2. A Search Algorithm*

- Most popular informed search method.
- Uses **$f(n) = g(n) + h(n)$**
 - $g(n)$: cost so far
 - $h(n)$: estimated cost to goal
- Combines optimality and efficiency.
- Example: GPS navigation systems use A* to calculate the shortest/fastest route.

3. Hill Climbing

- Always moves toward the direction that improves the heuristic value.
- Can get stuck in local maxima.
- Example: Used in machine learning optimization problems.

Industry Example

Food Delivery (like Swiggy/Zomato):

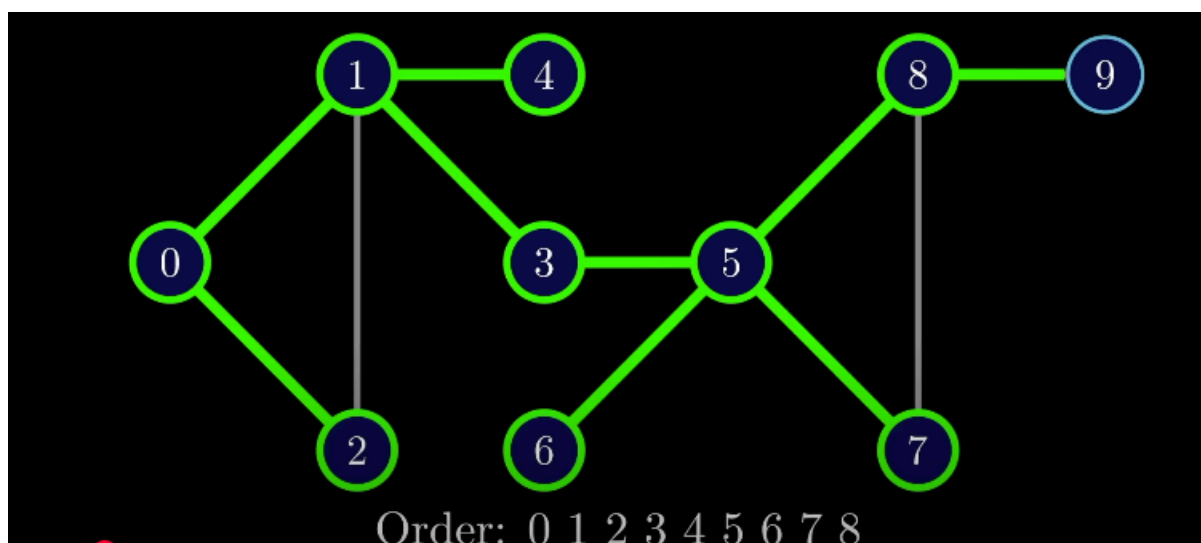
- **Initial state**: Restaurant location.
- **Goal state**: Customer's address.
- **Actions**: Possible delivery routes.
- **Uninformed search** → Delivery boy tries all roads one by one (time-consuming).
- **Informed search (A^*)** → System uses heuristics like “shortest distance” + “least traffic” to suggest the **fastest route**.

E-commerce Recommendation (Amazon/Flipkart):

- Instead of blindly showing all products, AI uses **heuristics (user history, ratings, preferences)** to recommend relevant items, saving time and improving efficiency.

Breadth First Search (BFS)

- Breadth-First Search (BFS) is one of the most important **uninformed (blind) search algorithms** in Artificial Intelligence. It explores the **state space level by level**, starting from the initial state and gradually expanding all neighboring nodes before moving deeper. This makes BFS systematic, complete, and useful for finding the **shortest path** when all actions have the same cost.
- Breadth-First Search (BFS) is simple and powerful. It guarantees both **completeness** and **optimality** for equal-cost problems. However, due to its high memory and time consumption, it is best suited for problems with **small state spaces** or where the shortest path is required.
- BFS always explores the shallowest (minimum depth) nodes first.



Steps of BFS-

1. Start from the **initial state**.
2. Put the starting node in a **queue** (FIFO – First In, First Out).
3. Expand the first node in the queue, and add all its children (neighbors).
4. Continue expanding nodes level by level until the **goal state** is found.
5. Stop when the goal is reached or when no nodes are left.

Key Properties

1. **Completeness**: BFS is complete; if a solution exists, BFS will definitely find it.
2. **Optimality**: If all step costs are equal, BFS always finds the shortest path.
3. **Time Complexity**: $O(b^d)$, where b = branching factor, d = depth of solution. (Can be very high in large state spaces).
4. **Space Complexity**: $O(b^d)$ because it stores all nodes in memory.

BFS is Used in Real Life

1. **Navigation Systems (Google Maps, Uber, Ola)**:
 - o BFS explores all routes **level by level**.
 - o Guarantees finding the **shortest route** if all roads have the same cost.
2. **Social Networks (Facebook, LinkedIn)**:
 - o BFS finds the **shortest connection** (e.g., friend-of-a-friend).

- o Example: “How many connections away is person X?”
- 3. **Network Broadcasting (Wi-Fi, Internet):**
 - o BFS ensures that a message reaches all nodes (computers) in the network.
- 4. **Web Crawlers (Google Search Engine):**
 - o BFS is used to explore web pages level by level.
- 5. **AI in Games (Chess, Puzzle solving):**
 - o BFS helps to explore all possible moves level by level to guarantee a win in small problems.

Why We Use BFS?

- To always get the **shortest path** (if all step costs are equal).
- To ensure **completeness** (it will always find a solution if one exists).
- Simple and systematic (good for small to medium problems).

2. Depth First Search (DFS) in Artificial Intelligence

Depth First Search (DFS) is a **search strategy** used in Artificial Intelligence (AI) for exploring a state space or graph/tree. DFS go **TO deep as possible along one branch** before backtracking and exploring other alternatives.

It is like a person exploring a maze who always chooses the first path available and continues until reaching the end, then backtracks if stuck, and tries another path.

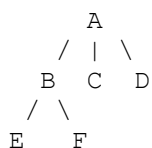
Working Principle of DFS

1. Start at the **initial node (root)**.
2. Explore the **first unvisited child**.
3. Continue moving **deeper** into the tree/graph until:
 - o The **goal state** is found, or
 - o A dead end is reached.
4. If a dead end occurs, **backtrack** to the last node with unvisited children and continue.
5. Repeat until the **goal state** is reached or all nodes are explored.

DFS is typically implemented using a **stack** (either explicitly with a data structure or implicitly with recursion).

Example

Consider this tree:



DFS order: **A → B → E → F → C → D**

Explanation: Start from **A**, go to **B**, then **E** (deepest). Once no further nodes are left, backtrack to **B**, then move to **F**, backtrack again, then explore **C**, and finally **D**.

Key Characteristics of DFS

- **Completeness**: Not guaranteed (may get stuck in infinite paths if depth is unbounded).
- **Optimality**: Not guaranteed (doesn't always find the shortest path).
- **Time Complexity**: $O(b^m)$, where
 - o b = branching factor (average children per node)
 - o m = maximum depth.

Where DFS is Used?

1. **Pathfinding in mazes or puzzles** → Explore deeply until exit is found.
2. **Scheduling tasks** → Used in job scheduling and dependency resolution.
3. **Compilers** → For parsing and syntax tree traversal.
4. **AI Games** → To explore all possible future moves deeply (though often improved with heuristics).
5. **Network Traversal** → Used in crawling large web structures or social networks.

Why DFS?

- **Memory efficient** compared to BFS (only stores one path at a time).
- Useful when the **solution is deep** in the tree.
- Easier to implement with recursion.

Real-Life Uses of DFS (Depth First Search)

1. **File/Folder Search in Computers**
 - o Suppose you open **C:/ drive** and go into **Documents** → **Projects** → **AI** → **Assignments**.
 - o You are going **deep into one folder** before checking others.
 - o This is exactly how DFS works.
2. **Maze Solving / Pathfinding**
 - o Imagine you are in a **maze**.
 - o You always choose the **first path** and keep walking until you reach a dead end.
 - o If blocked, you **backtrack** to the last decision point and try another path.
 - o DFS helps in such exploration.
3. **Social Network Friend Suggestions**
 - o Platforms like **Facebook/LinkedIn** use DFS to explore friend connections.
 - o Example: From you → friend → friend of friend → and so on, before backtracking.
4. **AI in Games (like Chess)**

- o In chess, DFS explores **possible moves deeply** (e.g., 10 steps ahead).
- o It checks outcomes of one sequence of moves fully, then backtracks and tries another.

5. Scheduling & Task Dependencies

- o In **project management** (like building software), some tasks depend on others.
- o DFS is used to find an order of execution → this is called **topological sorting**.
- o Example: You cannot “compile code” before “writing code.”

6. Network Routing / Crawling Websites

- o Web crawlers use DFS to explore web pages.
- o Start at one site → follow a link → then another → and so on until depth is reached.

Why DFS in Real Life?

- Memory efficient (uses less space than BFS).
- Good for **deep solutions**.
- Simple to implement (recursion/stack).

3. Depth Limited Search (DLS)

Depth Limited Search (DLS) is a variation of Depth First Search (DFS) where the search is restricted to a fixed depth limit. Unlike normal DFS, which may continue indefinitely in deep or cyclic graphs, DLS prevents infinite exploration by stopping when the predefined depth is reached. If the goal node arrives within this limit, DLS successfully finds it; otherwise, the algorithm terminates with a **cutoff** result. This makes DLS useful in controlling time and space complexity, but if the solution is deeper than the limit, it will not be found, requiring careful choice of the depth value.

Let suppose you studying a language and going to deep and deep but ignore the result and solution.

Characteristics of Depth Limited Search (DLS)

1. **Depth Bounded:** Expands nodes only up to a predefined depth.
2. **DFS-Based:** Works like Depth First Search but with a limit.
3. **Memory Efficient:** Requires less memory compared to BFS (stores only current path).
4. **Cutoff Handling:** If depth limit is too small, the algorithm reports **cutoff failure**.
5. **Incomplete:** May miss solutions that are deeper than the depth limit.
6. **Useful for Infinite Graphs:** Prevents going indefinitely in cyclic or infinite paths.

Why We Use DLS

- To **avoid infinite loops** in large or cyclic search spaces.

- To **control resource usage** (time and memory).
- To **test solutions within a depth range** before going deeper.
- As a **step toward Iterative Deepening Search**, which combines advantages of BFS and DFS.

Where DLS is Used in Industry

1. AI Planning & Robotics

- Robots exploring environments often set a depth limit (steps or moves) to avoid wasting time on very long or impractical paths.

2. Game Playing (e.g., Chess, Go)

- In adversarial search, game trees are explored only up to a certain depth (ply limit).
- Example: In chess engines, DLS explores moves up to depth 3, then evaluates with heuristics.

3. Route Planning / Navigation

- Maps or GPS systems may restrict search depth to avoid impractical routes.

4. Problem-Solving Agents

- Used in AI agents where the solution is expected within a certain number of steps.

5. Decision-Making Systems

- Helps in scenarios where computation time is critical (e.g., embedded systems).

4. Iterative Deepening Search (IDS)

Iterative Deepening Search (IDS) is a search strategy that combines the space efficiency of Depth First Search (DFS) with the completeness and optimality of Breadth First Search (BFS). It works by repeatedly applying **Depth Limited Search (DLS)** with increasing depth limits until the goal is found.

This process continues until the goal node is discovered. Although some nodes are generated multiple times, the overhead is small compared to the benefits.

Key Features

- **Complete:** Always finds a solution if one exists.
- **Optimal:** Finds the shallowest solution, like BFS.
- **Memory Efficient:** Uses memory proportional to DFS ($O(d)$, where d is solution depth).

Usage of IDS

- To get the **optimal solution** without the heavy memory cost of BFS.
- To **avoid infinite loops** seen in DFS on cyclic graphs.
- To strike a balance between **completeness, optimality, and space efficiency**.
- To explore **large or infinite state spaces** effectively.

Informed (Heuristic) Search Strategies

Uninformed (blind) search algorithms like BFS, DFS, UCS do not use any extra knowledge about the problem.

In contrast, **Informed Search (Heuristic Search)** uses additional information (called a **heuristic**) to guide the search process towards the goal more efficiently.

- **Heuristic ($h(n)$)** = an estimate of the cost from the current state (node n) to the goal state.
- Informed search $\rightarrow$ faster, less memory compared to blind search.

1. Greedy Best-First Search (GBFS)

Greedy Best-First Search (GBFS) is an **informed search algorithm** that uses a **heuristic function ($h(n)$)** to decide which node appears to be closest to the goal.

In The “greedy” algorithm its always select the best option **at the moment** — it tries to reach the goal as quickly as possible, without considering the total cost.

This method makes GBFS **fast**, but not always **accurate**. It may find a solution quickly, but not necessarily the **shortest or optimal path**, because it focuses only on the estimated distance to the goal and ignores the actual path cost.

Heuristic Function

The heuristic function **$h(n)$** estimates how close a node **n** is to the goal.

- **$h(n)$** = estimated cost from the current node to the goal.
Example: In a map navigation problem, **$h(n)$** can be the **straight-line distance** (Euclidean distance) from city n to the destination city.

A smaller value of **$h(n)$** means the node is *assumed* to be closer to the goal, so GBFS selects it first.

Algorithm Steps

1. Start from the **initial node** and put it into a **priority queue**.
2. Select the node with the **lowest heuristic value $h(n)$** (i.e., the most promising node).
3. Expand this node to generate all possible successors.
4. For each successor, calculate its **$h(n)$** .
5. Add all successors to the queue, sorted by **$h(n)$** .
6. Repeat the process until the **goal node** is reached or the queue becomes empty.

Advantages

- **Faster** than other search methods when the heuristic is good.
- **Memory-efficient** for large problems.
- Ideal for **real-time or quick decision-making** tasks.

Disadvantages

- **Not optimal:** It may miss the shortest path.
- **Incomplete:** It can get stuck in loops or dead ends.
- Strongly **depends on the quality** of the heuristic.
- Maybe give you best solution but not sure always give you best solution.

Real-Life Applications

- GPS and route-finding systems for approximate routes.
- Robot navigation where speed is more important than accuracy.
- Game AI for chasing or moving toward targets efficiently.

Example

Imagine a **delivery drone** finding a path from a warehouse (A) to a customer's home (G). GBFS will always choose the direction that **seems** closest to G (based on straight-line distance), even if that route ends up being longer in real travel time. Thus, GBFS can find a quick but not necessarily optimal route.

2. *A (A-Star) Search Algorithm**

The **A*** (pronounced "A-star") search algorithm is one of the most popular and powerful **informed search algorithms** in Artificial Intelligence. It combines the advantages of both **Uniform Cost Search** and **Greedy Best-First Search** to find the **optimal and efficient path** from the initial state to the goal state.

A* uses both the **actual cost** of reaching a node and the **estimated cost** to reach the goal from that node, making it both **complete** (guaranteed to find a solution if one exists) and **optimal** (finds the shortest possible path).

Main Formula

A* evaluates every node using the function:

$$f(n) = g(n) + h(n)$$

Where:

- **g(n)** = the cost of the path from the start node to node *n* (actual cost).
- **h(n)** = the estimated cost from *n* to the goal (heuristic).
- **f(n)** = total estimated cost of the cheapest solution path passing through *n*.

Algorithm Steps

1. Start with the **initial node**, and enter the starting node.
2. If open list is empty then return fail.
3. Place it into a **priority queue** (called the open list).
4. Select the node from open list which has smallest value $g(n) + h(n)$.
5. If this node is the **goal**, stop — solution found.
6. Otherwise, **expand** the node (generate successors).
7. For each successor, calculate its **$g(n)$** , **$h(n)$** , and **$f(n)$** values.
8. Add successors to the open list if they're not already visited or have a better $f(n)$ value.
9. Repeat until the goal node is selected.

Example

In a **Google Maps** navigation system:

- **$g(n)$** = actual distance traveled so far.
 - **$h(n)$** = straight-line (air) distance to the destination.
- A* combines both to find the **fastest and shortest route** efficiently.

Advantages

- **Optimal and complete** if the heuristic is admissible (never overestimates the cost).
- **Efficient** – explores fewer nodes than uninformed searches.
- **Widely applicable** to various real-world pathfinding problems.

Disadvantages

- **Memory-intensive** – stores all generated nodes.
- **Performance depends** on the quality of the heuristic function.

Real-Life Applications

- GPS and map routing systems.
- Game development for intelligent movement and pathfinding.
- Robotics navigation and planning.
- Network optimization and logistics.

3. Heuristic Functions (Admissibility & Consistency)

- **Heuristic Function ($h(n)$):**
 - **Heuristic** used to estimate the cost from current node to the goal.
 - Example: In map navigation, heuristic = straight-line distance from current location to destination.
- **Admissibility:**
 - A heuristic is **admissible** if it never overestimates the actual cost to reach the goal.
 - Ensures **optimality** of A*.

- Sudoku solver: ek possible number fill karo, agar sahi nahi → change karo, repeat until solved.
- Hill = solution value, higher hill = better solution.

4. Local and Global Search Algorithms

Local Search Techniques

Local Search Techniques are problem-solving methods used in Artificial Intelligence to find solutions by exploring the search space **locally**, by moving from one potential solution to a neighbouring one. Unlike systematic search algorithms such as Breadth-First or Depth-First Search, local search does not maintain or explore the entire search tree. Instead, it focuses on improving a single current solution through small, incremental changes.

These techniques are especially useful for **optimization problems** where the goal is to find the best possible solution among many options. Local search operates on the principle of evaluating neighboring states and moving toward states that provide better performance according to a defined **objective or fitness function**.

In local search techniques we used different type of search method:

Hill Climbing,

Simulated Annealing,

Genetic Algorithms.

Common local search methods include **Hill Climbing, Simulated Annealing, Genetic Algorithms**. These techniques can handle large or continuous search spaces efficiently, though they may sometimes get trapped in local optima.

Local search is widely used in **engineering design, scheduling, machine learning, robotics, and route optimization** because of its simplicity, low memory requirement, and adaptability to complex real-world problems. It provides approximate but often highly effective solutions in cases where exhaustive search is impractical or computationally expensive.

Advantages:

- Less memory usage
- Simple to implement

Disadvantages:

- Can get stuck in **local maxima/minima** or **plateaus**.

2. Hill Climbing

Hill Climbing is a **local search optimization algorithm** used in Artificial Intelligence to find the best or optimal solution by iteratively improving a current state. It is inspired by the idea of climbing to the top of a hill, where each step takes the system to a higher (better) state according to an **evaluation or heuristic function**.

The algorithm starts with an initial random solution and repeatedly moves to a neighbouring solution that has a higher value of the evaluation function. The process continues until no better neighboring state exists, indicating that a peak (maximum point) has been reached. Hill Climbing is a **greedy algorithm**, meaning it makes decisions that seem best at the current step without considering future consequences.

However, this approach can lead to problems such as:

1. **Local Maximum:** The algorithm may stop at a peak that is not the global maximum.
2. **Plateau(pla-toh):** The algorithm may reach a flat region where neighbouring states have the same value.
3. **Ridge:** The path to the global maximum may require moving in a direction that initially seems worse.

Hill Climbing is widely used in **function optimization, robotics (path planning), scheduling, and game playing**, where a precise global optimum is less important than a good approximate solution.

+Hill Climbing is simple, fast, and memory-efficient but can fail to find the global optimum due to local traps, making it best suited for problems where approximate solutions are acceptable.

3. Simulated Annealing (SA)

Simulated Annealing is a **probabilistic local search optimization technique** used in Artificial Intelligence to find near-optimal solutions in a large search space. It is especially useful in situations where traditional local search methods like **Hill Climbing** get trapped in **local maxima or minima** and fail to reach the **global optimum**. The technique is inspired by the **annealing process in metallurgy**, where metal or glass is heated to a high temperature and then slowly cooled. This controlled heating and cooling allow particles to move freely at first and gradually settle into a **stable, low-energy state**.

In Simulated Annealing, we apply the same concept to problem-solving. Initially, the algorithm starts with a **high temperature** and an **initial solution**. At high temperature, the algorithm is allowed to **accept even worse solutions** intentionally. This is important because accepting worse solutions during the early phase helps the algorithm **escape from local optima**. As the algorithm progresses, the temperature is slowly decreased. With decreasing temperature, the algorithm becomes more selective and accepts mostly **better solutions**. Eventually, when the temperature becomes very low, the system stabilizes and reaches a final solution, ideally close to the **global optimum**.

The acceptance of a worse move is controlled by a **probability function**, usually based on the **Boltzmann distribution**:

$$P = e^{-\frac{\Delta E}{T}}$$

Where:

- ΔE = difference in cost or energy between current and new solution
- T = current temperature
- e = mathematical constant

If the new solution is worse, it may still be accepted with a probability that decreases as the temperature lowers. This is the key factor that differentiates Simulated Annealing from other local search methods.

Key Steps in Simulated Annealing

1. Start with an initial solution and a high temperature.
2. Generate a **neighbor solution** by making a small random change to the current solution.
3. Calculate the change in cost.
4. If the new solution is better, accept it.
5. If the new solution is worse, accept it with some probability depending on temperature.
6. Slowly **decrease** the temperature based on a cooling schedule.
7. Stop when the system reaches a stable state.

Example

Suppose you want to visit **10 tourist places in a city** in minimum travel distance. Starting from a random route:

- At the beginning, you try different random route combinations, even if they look bad (high temperature phase).
- Gradually, you reduce the randomness and prefer only shorter routes (cooling phase).
- Eventually, you settle on a route that is **close to the shortest possible** (final solution).

Advantages

- Can escape local optima.
- Works well in large, complex search spaces.

Limitations

- Performance depends heavily on the **cooling schedule**.
- Slow convergence if cooling is too gradual.

4. Genetic Algorithms (GA)

English Explanation

- GA is inspired by **Darwin's evolution theory** ("survival of the fittest").
- Key components:
 - o **Population** = set of candidate solutions
 - o **Selection** = choose best solutions
 - o **Crossover** = combine parts of solutions to create new ones
 - o **Mutation** = randomly change part of solution to maintain diversity
- Iteratively, the population evolves toward **better solutions**.
- Excellent for **complex, multimodal optimization problems**.

GA = population of solutions evolve hoti hai → selection, crossover, mutation se **best solution nikalta hai**.

Example

- Breeding dogs or crops for best traits.
- Online recommendation system: evolving algorithms to give better suggestions.

5. Global Search Algorithms for Optimization Problems

Global Search Algorithms are a class of problem-solving techniques used in **optimization** to find the **best possible solution** from a very large set of possible solutions. Unlike **local search algorithms**, which only explore neighboring solutions and may get trapped in **local optima**, global search algorithms explore the **entire search space** more broadly. Their goal is to avoid local traps and move toward the **global optimum**, which represents the most optimal solution of the whole problem.

Optimization problems appear in various fields such as machine learning, network routing, engineering design, data mining, timetable scheduling, and many more. These problems involve maximizing or minimizing a specific **objective function**. For example, minimizing cost, maximizing accuracy, or minimizing travel distance.

Global search algorithms do not rely solely on the current solution. Instead, they use strategies like **randomness**, **population-based evolution**, **probability control**, or **memory of past states** to explore multiple areas of the search space. This helps them avoid getting stuck at suboptimal solutions.

1. Genetic Algorithms (GA)

Genetic Algorithms are inspired by the process of **natural selection** in biology. They maintain a **population of candidate solutions**, where each candidate is considered an "individual." These individuals undergo operations like:

- **Selection** (choosing fit individuals),
- **Crossover** (combining two solutions),
- **Mutation** (making random modifications).

Over multiple generations, the population evolves toward better solutions. GA is especially useful in **non-linear** and **complex search spaces**, such as feature selection, scheduling, and design optimization.

2. Simulated Annealing (SA)

Simulated Annealing is inspired by the **annealing process in metallurgy**. The idea is to start with a high “temperature” that allows exploring even lower-quality solutions. As temperature decreases gradually, the algorithm becomes more selective. This helps the search escape local optima and move toward a **global optimum**.

3. Particle Swarm Optimization (PSO)

Particle Swarm Optimization is based on the **collective behavior** of birds flocking or fish schooling. Each solution in PSO is called a **particle**, and all particles move through the search space while adjusting their positions based on:

- Their own best experience
- The best position found by the swarm

This group-based learning helps converge to an optimal global solution. PSO is widely used in **neural network training**, parameter tuning, and engineering optimization.

Global Search Algorithms are powerful and flexible techniques capable of navigating **large and complex search spaces** to find high-quality solutions. By using randomness, evolution, collaboration, or memory, they overcome the limitations of local search and are widely used in modern optimization tasks across science, engineering, and artificial intelligence.

Global optimization = **sabse best solution dhoondhna**, na ki local peak me atakna.

Example

- Airline ticket pricing: optimize globally to maximize profit.
- Google Maps: find fastest route among all possible roads, not just nearby shortcuts.

Minimax Algorithm in Artificial Intelligence

The **Minimax Algorithm** is one of the most important concepts in **Artificial Intelligence (AI)**, especially in **game playing and decision-making systems**.

The name “*Minimax*” comes from the idea of **minimizing the possible loss** in the worst-case scenario. One player (called **MAX**) tries to **maximize** their chance of winning, while the opponent (called **MIN**) tries to **minimize** it.

It is used in games that involve **two players** — one trying to **maximize** their score and the other trying to **minimize** it. **Typical examples include games like Chess, Tic-Tac-Toe, and Checkers.** The goal of this algorithm is to make the **best possible move** for the player, assuming that the **opponent also plays optimally**.

The algorithm explores **all possible moves** of both players by building a **game tree**, which represents every possible move and counter-move up to a certain depth.

At each level of the tree:

- The **MAX node** represents the player's turn — it chooses the **maximum value** among its children.
- The **MIN node** represents the opponent's turn — it chooses the **minimum value** among its children.

How It Works (Step-by-Step)

1. **Generate the Game Tree:**
All possible future moves from the current position are represented in a tree structure.
2. **Assign Scores to Terminal Nodes:**
At the end of the game (win, lose, draw), values are assigned:
 - o Win → +1
 - o Lose → -1
 - o Draw → 0
3. **Backpropagate the Scores:**
Starting from the terminal nodes, scores are passed upward:
 - o At **MIN nodes**, choose the *minimum* value (opponent's best move).
 - o At **MAX nodes**, choose the *maximum* value (player's best move).
4. **Choose the Best Move:**
The root node (current state) will have the best possible move chosen for the MAX player based on these values.

Think of it like **planning a chess move**: before moving a piece, you imagine how your opponent will respond. You pick a move that keeps you in the best possible position even after the opponent's smartest response.

Advantages

- Provides an **optimal strategy** for deterministic, turn-based games.
- Ensures the player never makes a move that leads to a guaranteed loss.

Limitations

- Very **slow** for large games like chess, since the number of possible moves grows exponentially.
- Requires optimization techniques like **Alpha-Beta Pruning** to speed up search.

The Minimax Algorithm forms the foundation of AI game-playing strategies. By simulating all possible outcomes and always assuming an intelligent opponent, it ensures that every move made is **logically the best**. In short, Minimax allows machines to “**think ahead**” and make smart, strategic decisions — just like a skilled human player.

Alpha-Beta Pruning

Pruning means **cutting off or eliminating branches** of the game tree that **do not need to be explored**, because they will not affect the final decision. The most commonly used pruning technique is **Alpha-Beta Pruning**, which works along with the Minimax algorithm. It reduces the number of nodes that must be evaluated, without changing the final outcome of the algorithm.

The need for pruning arises due to the following reasons:

1. **Reduces Time Complexity:**
Without pruning, the algorithm may take too long to analyze even a few moves ahead. Pruning helps skip unnecessary calculations, allowing the algorithm to think faster.
2. **Reduces Memory Usage:**
Storing and processing large game trees requires a lot of memory. Pruning limits the size of the explored tree, saving storage.
3. **Allows Deeper Search:**
Since pruning frees up time, the algorithm can explore **greater depth** in the same time frame. This results in **better decision-making** and stronger gameplay.
4. **Makes Real-Time Games Possible:**
In competitive games, decisions must be made quickly. Pruning helps AI respond in real time, making it practical for live gameplay.

In summary, pruning is essential for **efficiency**. It ensures that the AI focuses only on **relevant moves**, reducing computation while still guaranteeing optimal results.

Working of Alpha-Beta Pruning

Alpha-Beta Pruning is an improved version of the Minimax Algorithm used for decision-making in two-player adversarial games such as Chess, Tic-Tac-Toe, or Checkers. The main purpose of Alpha-Beta Pruning is to reduce the number of nodes explored in the game tree without affecting the final result. In simple words, it helps the computer decide faster by ignoring those branches of the search tree that **cannot influence the final decision**.

Basic Idea

Minimax works by generating a full game tree and evaluating every possible move. However, evaluating the entire tree is time-consuming and computationally expensive. Alpha-Beta Pruning adds two parameters, **Alpha** and **Beta**, which act as boundaries to decide when to stop evaluating certain branches.

- **Alpha (α)** = The **maximum score** that the maximizing player (MAX) can guarantee so far.
- **Beta (β)** = The **minimum score** that the minimizing player (MIN) can guarantee so far.

During the traversal of the tree:

- The **MAX player** tries to increase the value (maximize the score).
- The **MIN player** tries to decrease the value (minimize the score).

If at any point it is found that the current move is **worse than the previously explored alternative**, there is no need to explore further, and that branch is **pruned** (ignored).

Step-by-Step Working

1. **Start with Minimax Algorithm:**
The tree is explored level by level. At MAX levels, we try to find the maximum value; at MIN levels, we try to find the minimum value.
2. **Initialize $\alpha = -\infty$ and $\beta = +\infty$:**
These values get updated as the algorithm progresses.
3. **Traverse the tree depth-first:**
This saves memory and helps to update alpha and beta values as soon as possible.
4. **At MAX Nodes:**
 - o Compare child values with α .
 - o If a value is greater than α , update α .
 - o If $\alpha \geq \beta$, **stop exploring siblings**, because MIN player will not allow MAX to reach that branch → **PRUNE**.
5. **At MIN Nodes:**
 - o Compare child values with β .
 - o If a value is less than β , update β .
 - o If $\beta \leq \alpha$, **stop exploring siblings**, because MAX player will not choose that branch → **PRUNE**.

In game-playing Artificial Intelligence, algorithms such as **Minimax** explore a **game tree** to determine the best possible move. However, game trees of real-world games like **Chess** and **Checkers** are extremely large, containing millions or even billions of possible states. Searching the entire game tree is often **impractical**, because it requires a huge amount of **time** and **computational power**. This is where **Pruning** becomes necessary.

Knowledge Representation and Reasoning (KRR)

Knowledge Representation and Reasoning (KRR) is a fundamental area of Artificial Intelligence (AI) concerned with how knowledge can be represented symbolically and how automated systems can reason about it. The goal of KRR is to enable machines to behave intelligently by storing, organizing, and manipulating information in a form that computers can process to draw meaningful conclusions.

In simple terms, KRR allows AI systems to “think” — by using stored information (knowledge) and applying logical rules (reasoning) to solve problems, make predictions, or understand situations. The efficiency and intelligence of an AI system largely depend on how well the knowledge is represented and how effectively the reasoning process is carried out.

Knowledge Representation and Reasoning (KRR)

Knowledge Representation and Reasoning (KRR) is a fundamental area of Artificial Intelligence (AI) concerned with how knowledge can be represented symbolically and how automated systems can reason about it. The goal of KRR is to enable machines to behave intelligently by storing, organizing, and manipulating information in a form that computers can process to draw meaningful conclusions.

In simple terms, KRR allows AI systems to “think” — by using stored information (knowledge) and applying logical rules (reasoning) to solve problems, make predictions, or understand situations. The efficiency and intelligence of an AI system largely depend on how well the knowledge is represented and how effectively the reasoning process is carried out.

2.6.1 Types of Knowledge

In Artificial Intelligence, knowledge is not uniform; it can be classified into several types based on its nature and use.

1. Declarative Knowledge

Declarative knowledge refers to “**knowing that something is the case.**”

It describes facts and things that are true in the world. For example:

- “Paris is the capital of France.”
- “Water boils at 100°C.”

This type of knowledge is represented in the form of statements or propositions. AI systems store declarative knowledge in databases or knowledge bases and use reasoning methods to infer new facts.

Example in AI:

An expert system for medical diagnosis stores declarative knowledge such as “If the patient has fever and cough, then the disease might be flu.”

2. Procedural Knowledge

Procedural knowledge refers to “**knowing how to do something.**”

It is related to methods, processes, and sequences of actions.

Solving mathematical equations or programming a robot’s movement. In AI, it is encoded as algorithms or rules guiding problem-solving and task execution efficiently.

For example:

- Knowing how to drive a car.
- Knowing how to solve a mathematical problem.

This knowledge is often represented as rules, algorithms, or procedures.

Example in AI:

In robotics, procedural knowledge tells a robot how to navigate from one point to another using obstacle avoidance algorithms.

3. Heuristic Knowledge

Heuristic knowledge is **experience-based knowledge** or “**rules of thumb.**”

It is not always accurate but provides quick, practical solutions to complex problems.

Example:

In chess, a heuristic rule might be “Control the center of the board early in the game.”

In AI:

Heuristics help in speeding up problem-solving and are used in search algorithms like A* (A-star) or Hill Climbing.

4. Meta-Knowledge

Meta-knowledge means “**knowledge about knowledge.**”

It involves understanding how to organize, control, and manipulate other types of knowledge.

In AI, meta-knowledge enables systems to decide which reasoning or inference methods are most appropriate in a given situation, improving flexibility and problem-solving capability.

Example in AI:

An expert system may have meta-rules that tell it when to use certain inference strategies.

5. Structural Knowledge

Structural knowledge defines **how different concepts or pieces of knowledge are related** to each other.

It helps in building relationships among objects and concepts using hierarchies, networks, or ontologies.

Example:

In biology, “A cat is a type of mammal” defines a hierarchical structure.

Semantic Networks

A **Semantic Network** is a graphical representation of knowledge in the form of **nodes** (concepts) and **links** (relationships between concepts).

It was one of the earliest methods of knowledge representation used in AI.

It organizes knowledge in a connected structure, allowing inheritance and logical associations, which help AI systems in understanding, reasoning, and natural language processing.

Each node represents an object or concept (like “bird,” “animal,” or “fly”), and each link represents a relationship (like “is a,” “has,” “can”).

Thus, a semantic network helps visualize how knowledge is connected.

Example

Consider the following knowledge:

- A sparrow is a bird.
- A bird is an animal.
- Birds can fly.

The corresponding semantic network would show:

Sparrow → is a → Bird → is a → Animal

Bird → can → Fly

Advantages

- Easy to visualize and understand.
- Efficient for inheritance: if “Birds can fly,” then “Sparrows” automatically inherit this property.
- Useful in natural language processing and expert systems.

Disadvantages

- Limited to simple relationships.
- Difficult to represent complex or uncertain knowledge.
- Lacks standardization for reasoning.

2.6.3 Frames

A **Frame** is a structured representation of knowledge that divides information into **slots** (attributes) and **fillers** (values).

It was introduced by Marvin Minsky in 1974 to represent stereotyped situations or real-world objects.

Each frame represents an entity, situation, or concept, similar to a database record. Frames can inherit properties from other frames, forming a hierarchy.

Example

A frame for **Car** might look like:

Slot	Filler
------	--------

Type	Sedan
------	-------

n

Color Red

Engine Petrol

Wheels 4

A specific car (say, “MyCar”) can inherit all properties from the **Car** frame but can override specific attributes (e.g., color = blue).

Advantages

- Hierarchical and organized.
- Supports inheritance (reduces redundancy).
- Closer to human-like representation of knowledge.

Disadvantages

- Difficult to represent dynamic or changing knowledge.
- Requires careful design to avoid inconsistency.

Application

Frames are widely used in:

- **Expert Systems**: representing knowledge about medical diseases or technical problems.
- **Robotics**: describing objects and environments.
- **Natural Language Understanding**: interpreting sentence meaning based on context frames.

2.6.4 Ontologies

An **Ontology** is a **formal and structured representation of knowledge** that defines the **concepts, relationships, and categories** of a particular domain.

It provides a shared and common understanding of information that can be communicated between humans and machines.

Ontologies are expressed using languages like **OWL (Web Ontology Language)** or **RDF (Resource Description Framework)** in AI systems.

Components of Ontology

1. **Concepts (Classes)**: Represent objects or ideas (e.g., Car, Engine, Person).
2. **Relations**: Define how concepts are connected (e.g., “hasPart,” “isA”).

3. **Instances:** Specific examples of concepts (e.g., “Honda City” is an instance of Car).
4. **Axioms:** Logical statements that define rules and constraints.

Example

In a **Healthcare Ontology**, concepts could include:

- *Disease, Symptom, Treatment, Doctor*
Relationships:
- *Disease has Symptom, Doctor treats Disease, Treatment cures Disease.*

Advantages

- Promotes data sharing and reuse.
- Enables reasoning and inference.
- Provides consistency and interoperability among AI systems.

Applications in AI and Industry

- **Semantic Web:** Ontologies enable intelligent data linking on the web.
- **Healthcare:** Representing diseases, symptoms, and treatments.
- **E-commerce:** Product categorization and recommendation.
- **Robotics:** Object recognition and task planning.

Knowledge Representation and Reasoning (KRR) is a key foundation of Artificial Intelligence. It enables machines to represent information about the world and reason logically to solve problems.

- **Types of Knowledge** define *what* kind of information is stored.
- **Semantic Networks** show *relationships* among concepts visually.
- **Frames** organize information into *structured templates*.
- **Ontologies** standardize *domain knowledge* for reasoning and sharing.

Together, these methods make AI systems more intelligent, logical, and human-like in understanding and decision-making.

2.8 First Order Predicate Logic (FOPL)

First Order Predicate Logic (FOPL) is an advanced form of logic used in Artificial Intelligence (AI) to represent complex statements about objects, their properties, and relationships among them.

While **Propositional Logic** only deals with simple true/false statements, **Predicate Logic** allows us to describe *how things relate* to each other in the real world.

For example:

- Propositional logic: “Ravi is a student” $\rightarrow$ can only be True or False.
- Predicate logic: “Ravi is a student in college X” $\rightarrow$ includes *object* (Ravi), *property* (student), and *relationship* (in college X).

This allows AI systems to reason more precisely and represent real-world knowledge in a structured, logical way.

2.8.1 Predicates, Variables & Quantifiers

Predicates

A **predicate** is a statement or property that describes something about an object or the relationship between objects.

It acts like a *function* that becomes true or false depending on the values given to its arguments.

Example:

- $\text{Student}(\text{Ravi}) \rightarrow$ means “Ravi is a student.”
- $\text{Teaches}(\text{Professor}, \text{Subject}) \rightarrow$ means “Professor teaches Subject.”

Predicates help AI express meaningful statements rather than simple facts.

Variables

Variables represent objects whose values are not fixed or specific.

They act as placeholders that can be replaced by actual objects during reasoning.

Example:

- $\text{Loves}(x, y) \rightarrow$ means “x loves y.”
Here, x and y are variables which can take any values such as $x = \text{Ravi}$, $y = \text{Music}$.

Quantifiers

Quantifiers define the scope of variables in logical statements.

There are two types:

1. Universal Quantifier ($\forall$) — “For all”

- Denotes statements that are true for *every object* in the domain.
- Example:
 $\forall x \text{ Student}(x) \rightarrow \text{Studies}(x)$
Meaning: “All students study.”

2. Existential Quantifier ($\exists$) — “There exists”

- Denotes that *at least one* object in the domain satisfies the statement.

- Example:
 $\exists x \text{ Teaches}(x, \text{AI})$
 Meaning: “There exists someone who teaches AI.”

Why FOPL is Important in AI

- Represents knowledge in structured form.
- Helps in logical reasoning and inference.
- Used in expert systems, natural language understanding, and theorem proving.

2.8.2 Horn Clauses

Definition

A Horn Clause is a special type of logical expression used in Predicate Logic and logic programming. It contains **at most one positive literal** and any number of negative literals. Horn clauses are written in the form:

A :- B₁, B₂, ..., B_n,

which means “A is true if all B₁, B₂, ..., B_n are true.”

They are the foundation of programming languages like **PROLOG**, where rules and facts are expressed using Horn Clauses. This structure makes logical inference and reasoning efficient, as it simplifies complex logical statements into rule-based forms for automated reasoning systems. A Horn clause can be written as:

P :- Q₁, Q₂, ..., Q_n

Which means “P is true if Q₁, Q₂, ..., Q_n are true.”

Types of Horn Clauses

1. **Fact:** A simple statement that is always true.
 Example: Human(Socrates).
2. **Rule:** Describes a relationship using implication.
 Example:
 Mortal(X) :- Human(X).
 (If X is human, then X is mortal.)
3. **Goal/Query:** A question asked to the knowledge base.
 Example: ?- Mortal(Socrates).

Real-Life Example:

If your knowledge base has:

Human(Socrates).

Mortal(X) :- Human(X).

And you ask the system “Is Socrates mortal?”,
The system checks and answers “Yes”, because the rule and fact logically prove it.

AI Use: Horn clauses are the backbone of **PROLOG**, **Expert Systems**, and **Inference Engines** where reasoning is done through rule chaining (forward or backward).

2.8.3 Unification

Unification is a fundamental process in First Order Predicate Logic used to make two logical expressions identical by finding a suitable substitution of variables. It determines how variables can be replaced with constants or other variables to make different predicates match. For example, Unify Knows(John, x) and Knows(John, Mary) → substitution {x/Mary}. Unification plays a vital role in **automated reasoning**, **theorem proving**, and **logic programming** languages like **PROLOG**. It allows inference engines to connect facts and rules systematically, enabling AI systems to perform logical deductions and derive meaningful conclusions from given knowledge. **Unification** is a process in logic and AI of *matching two logical expressions* by finding a suitable substitution for their variables so that they become identical.

It is like solving a puzzle — replacing unknowns with values to make both sides of a statement equal.

Example:

Suppose we have:

1. Knows(John, x)
2. Knows(John, Mary)

Here, if we substitute $x = \text{Mary}$, both become identical.

Hence, they are **unifiable** with the substitution {x/Mary}.

Use in AI:

Unification is a key process in **automated reasoning** and **logic programming**:

- Helps AI systems match rules with facts.
- Used in **resolution theorem proving**, **PROLOG inference**, and **pattern matching**.

Example:

When you search “teacher of AI” on Google, the system tries to “unify” your query with its knowledge base — matching your words with stored patterns like “professor teaches AI.”

This logical matching process is similar to **unification** in AI reasoning.

Summary Table

Concept	Meaning	Example	Used In
Predicate	Property or relation	Teaches(Prof, AI)	Knowledge representation
Variable	Placeholder for objects	x, y	Logic statements
Quantifier	Defines scope ($\forall$, $\exists$)	$\forall x \text{ Student}(x) \rightarrow \text{Studies}(x)$	Logical reasoning
Horn Clause	Rule with ≤ 1 positive literal	$\text{Mortal}(X) :- \text{Human}(X)$	PROLOG
Unification	Matching logical patterns	{x/Mary}	Theorem proving

FOPL gives AI systems the *power of human-like reasoning*.

It allows machines not only to *store facts* but also to *understand relationships, infer new knowledge, and answer logical queries*.

From **chatbots** to **expert systems**, **semantic search**, and **robotic decision-making**, FOPL serves as the mathematical foundation of intelligent reasoning.

2.9 Reasoning Techniques

Reasoning is the process by which Artificial Intelligence (AI) systems derive conclusions from known facts or data. It enables machines to simulate human-like thinking and decision-making by applying logical rules to a knowledge base. Two important reasoning techniques used in AI are **Forward Chaining** and **Backward Chaining**. Both are forms of inference mechanisms used in rule-based systems and expert systems.

2.9.1 Forward Chaining

Forward chaining is a data-driven reasoning approach. It begins with the known facts and applies inference rules to extract more data until a specific goal or conclusion is reached. The system moves “forward” from the initial facts toward the conclusion.

Explanation:

In forward chaining, the reasoning process starts from the available information in the knowledge base. The inference engine examines each rule to determine whether the given facts satisfy its condition (the *if* part of the rule). When the condition is met, the rule fires,

and the *then* part (conclusion) of the rule is added as a new fact. This process continues iteratively until the goal state is achieved or no new facts can be inferred.

Structure of Rule:

A typical rule in forward chaining follows the **IF–THEN** format:

IF condition (premise) THEN conclusion (action)

Steps in Forward Chaining:

1. Identify all facts that are currently true (known data).
2. Match these facts with the conditions of rules in the knowledge base.
3. If a rule's condition is satisfied, execute the rule and add the conclusion to the set of known facts.
4. Repeat the process until the goal or desired conclusion is reached.

Example:

Consider a medical diagnosis system:

- Rule 1: IF the patient has fever AND cough THEN the patient may have flu.
- Rule 2: IF the patient may have flu THEN prescribe rest and fluids.

If the facts are:

- The patient has fever.
- The patient has cough.

Then the system applies Rule 1 → concludes the patient may have flu.

Next, Rule 2 is triggered → concludes “prescribe rest and fluids.”

Example:

In an **automated fire alarm system**, sensors detect smoke and heat (facts). The system then applies rules:

- IF smoke is detected AND temperature is high THEN activate alarm.
Thus, the system moves from data (sensor readings) to action (alarm), showing how forward chaining is data-driven.

Advantages:

- Easy to implement and understand.
- Useful when the goal is unknown but data are available.
- Suitable for real-time and control-based systems.

Disadvantages:

- Inefficient if there are many rules but few relevant to the goal.

- Can generate unnecessary conclusions.

Applications:

- Medical diagnosis systems.
- Automated control systems.
- Sensor-based monitoring and decision-making.
- Business rule engines.

2.9.2 Backward Chaining

Backward chaining is a goal-driven reasoning approach. It starts with a hypothesis or goal and works backward to determine whether known facts support it. The system looks for rules that could lead to the goal and verifies whether their premises are true.

In backward chaining, the inference engine begins from the goal (the *then* part of a rule) and checks whether the corresponding conditions (the *if* part) are satisfied by the facts in the knowledge base. If the conditions are not directly satisfied, the system recursively sets sub-goals to verify those conditions. This process continues backward until all conditions are satisfied or disproved.

Structure of Rule:

IF condition (premise) THEN conclusion (goal)

In backward chaining, reasoning starts from the **conclusion** and attempts to verify the **premise**.

Steps in Backward Chaining:

1. Identify the goal (the statement to be proven true).
2. Search for rules whose conclusion matches the goal.
3. For each such rule, check if its premises are true.
4. If a premise is not known, make it a sub-goal and repeat the process.
5. If all premises are proven true, the goal is achieved.

Example:

In a student evaluation system:

- Rule 1: IF the student has 90% attendance AND score > 80 THEN the student is eligible for a scholarship.

Goal: “Is the student eligible for a scholarship?”

The system looks backward:

- Does the student have 90% attendance?
 - Is the score > 80 ?
- If both are verified true, the conclusion (eligibility) is proven.

Example:

In **criminal investigation**, if police start from a hypothesis like “John is the culprit,” they work backward — checking evidence, witness statements, and motives — to verify whether this hypothesis holds true. This is a clear analogy of backward chaining.

Advantages:

- Focuses only on relevant rules related to the goal.
- More efficient when the number of possible facts is large but goals are specific.
- Suitable for diagnostic and problem-solving systems.

Disadvantages:

- Not efficient when multiple possible goals exist.
- Requires complete and accurate knowledge base.

Applications:

- Expert systems (e.g., MYCIN in medical diagnosis).
- Troubleshooting and diagnostic tools.
- Legal reasoning and crime analysis.
- Decision-support systems.

Probabilistic Reasoning

Need of Probability in AI

Probability plays a fundamental role in Artificial Intelligence because real-world environments are uncertain, incomplete, and constantly changing. In many situations, an intelligent agent cannot rely on exact or deterministic information. Instead, it must make decisions by estimating the likelihood of various outcomes. Probability provides a mathematical framework that allows AI systems to represent uncertainty, update beliefs when new information is observed, and choose the most reasonable action even when complete knowledge is not available.

In fields such as speech recognition, medical diagnosis, weather forecasting, robotics, and machine learning, data is often noisy or ambiguous. Probability helps the system deal with this ambiguity by assigning numerical values to the chances of different events. It further

enables AI to combine prior knowledge with newly observed evidence to derive more accurate conclusions.

Example:

Consider the prediction of rain by a weather application. The system does not know with certainty whether it will rain tomorrow. Instead, it analyses past climate data, humidity levels, and cloud patterns to compute a probability such as “There is a 70% chance of rain.” This helps users plan their day even in the presence of uncertainty. Similarly, AI-based health apps estimate the probability of illness based on symptoms and medical data.

Bayesian Reasoning

Bayesian reasoning is a method of reasoning under uncertainty that uses Bayes’ Theorem to update beliefs based on new evidence. It begins with an initial belief or prior probability about an event. When new data becomes available, this prior belief is revised, producing a posterior probability. This updated probability reflects the most reasonable belief after combining old information with new evidence. Bayesian reasoning is widely used in AI because it provides a systematic and mathematically sound approach to learning from observations.

The power of Bayesian reasoning lies in its ability to incorporate both historical knowledge and fresh data, allowing AI systems to improve their predictions over time. It is used in machine learning models, spam detection systems, expert systems, recommendation engines, and robotics, where continuous learning and adaptation are required.

Example:

Consider a medical diagnosis scenario. Suppose a doctor initially believes there is a small chance that a patient has a particular disease (prior probability). When the patient undergoes a medical test and the result is positive, the doctor updates the belief by applying Bayes’ Theorem. The revised probability (posterior probability) becomes much higher because the evidence strongly supports the disease. AI-based diagnostic tools follow the same approach to guide decision-making.

Conditional Probability

Conditional probability refers to the likelihood of an event occurring given that another related event has already occurred. It is written as $P(A|B)$, meaning “the probability of event A occurring given event B has occurred.” This concept is essential in Artificial Intelligence because many decision-making tasks depend on relationships between events rather than isolated probabilities. Conditional probability allows AI systems to model dependencies, predict outcomes, and reason logically in uncertain environments.

It is particularly important in classification tasks, natural language processing, recommendation systems, expert systems, and decision-making models. By understanding how one event influences another, AI can make more accurate predictions. Conditional probability also forms the foundation of Bayesian reasoning and probabilistic graphical models.

Example:

Imagine you want to know the probability that a student will score high in an exam given that they have studied for more than six hours. The probability of scoring high increases significantly when the condition (studying for six hours) is known. Mathematically, this is expressed as $P(\text{High Score} \mid \text{Studied Six Hours})$. AI systems such as YouTube recommendations work similarly: the probability of recommending a video increases if the system knows you have previously watched similar content. Thus, conditional probability helps in modeling real-life cause–effect relationships.

Bayesian Networks (Representation & Inference)

A Bayesian Network is a graphical model used to represent probabilistic relationships among a set of variables. It consists of nodes (representing variables) and directed edges (representing dependencies). Each node is associated with a Conditional Probability Table (CPT) that quantifies the effect of parent nodes on that variable. This structure allows Bayesian Networks to model complex real-world systems where uncertainty, causality, and interdependence exist.

Bayesian Networks are powerful tools in Artificial Intelligence because they support two major tasks: **representation** and **inference**. Representation organizes knowledge in a structured, graphical form, making relationships easier to understand. Inference involves computing probabilities of unknown variables based on known evidence. This helps in prediction, diagnosis, decision-making, and learning.

Bayesian Networks are used in medical diagnosis systems, fraud detection, weather prediction, expert systems, and robotics. They provide a clear way to model real-world uncertainty and reason about cause-and-effect relationships.

Example:

Consider a medical network with three variables: “Smoking,” “Lung Cancer,” and “Cough.” Smoking increases the probability of Lung Cancer, and Lung Cancer increases the probability of Cough. Using this network, if a patient reports coughing, the system can infer the likelihood of Lung Cancer and whether smoking might be a contributing factor. This structured reasoning helps AI systems make informed decisions.

Propositional Logic

Propositional Logic is a foundational branch of logic used for representing factual knowledge in the form of simple statements called propositions. Each proposition can only have one of two possible truth values: **True (T)** or **False (F)**. Propositional logic allows these propositions to be combined using logical connectives to form more complex expressions that represent real-world situations. The strength of propositional logic lies in its formal structure, which ensures that reasoning can be performed systematically and unambiguously.

Using propositional logic, AI systems can verify whether a conclusion logically follows from a set of premises. Its strict rule-based nature makes it suitable for early expert systems, decision systems, automated theorem proving, and many reasoning applications. Although propositional logic does not handle relationships between objects (that is done in First-Order Logic), it still acts as the basis for all higher-level logical systems. Through truth tables, inference rules, and the resolution method, propositional logic provides a complete framework for analyzing the correctness of arguments. Thus, it is one of the essential foundations of knowledge representation and reasoning in Artificial Intelligence.

Truth Table Example for a Simple Proposition (P)

2.7.1 Syntax and Semantics

Syntax in propositional logic defines the formal structure used to create valid statements. It specifies the symbols allowed and the rules for combining them. The main syntactic components are:

1. **Propositional variables** (P, Q, R...) representing simple statements.
2. **Logical connectives** such as NOT ($\neg$), AND ($\wedge$), OR ($\vee$), IMPLICATION ($\rightarrow$), and BICONDITIONAL ($\leftrightarrow$).
3. **Parentheses** for controlling grouping.

4. **Well-Formed Formula (WFF)**, which is any syntactically valid expression.

For example, $(P \wedge \neg Q) \rightarrow R$ is a well-formed formula.

Semantics defines the meaning of these syntactic expressions. It provides truth values to propositions and explains how connectives change or combine these values. Semantics tell us *whether a formula is true under a particular interpretation*. For example, if P is true and Q is false, semantics help determine the truth of $(P \vee Q)$ or $(P \rightarrow Q)$.

Syntax ensures correct structure; semantics ensures correct meaning. Together, they allow AI systems to evaluate, compare, and verify logical statements in a consistent manner.

Truth Table Example for Semantics ($P \wedge Q$)

	P	Q	$P \wedge Q$
	T	T	T
	T	F	F
	F	T	F
	F	F	F

2.7.2 Logical Connectives & Truth Tables

Logical connectives allow simple propositions to be combined into more complex expressions. Each connective has a well-defined behavior, represented using truth tables.

1. **NOT ($\neg P$):** Reverses the truth value of P.
2. **AND ($P \wedge Q$):** True only when both P and Q are true.
3. **OR ($P \vee Q$):** True when at least one of P or Q is true.
4. **IMPLICATION ($P \rightarrow Q$):** False only when P is true and Q is false; otherwise true.
5. **BICONDITIONAL ($P \leftrightarrow Q$):** True when P and Q share the same truth value.

These connectives form the building blocks for constructing logical expressions. Truth tables help analyze their behavior under all possible truth combinations, making them essential for evaluating formulas, proving equivalence, and performing reasoning in AI. Using connectives, complex real-world conditions can be represented symbolically, allowing AI systems to deduce conclusions logically and systematically.

Truth Tables

P	$\neg P$
T	F
F	T

NOT

AND

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

OR

P	Q	$P \vee Q$
T	T	T
T	F	T
F	T	T
F	F	F

IMPLICATION

P	Q	$P \rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

BICONDITIONAL

P	Q	$P \leftrightarrow Q$
T	T	T
T	F	F
F	T	F
F	F	T

7.3 Inference Rules

Inference rules are formal principles used to derive new statements from existing statements. They help determine what conclusions logically follow from given premises. Inference rules form the backbone of automated reasoning systems, theorem provers, and logic-based AI applications.

Inference rules allow AI systems to perform deduction — the process of deriving logically correct conclusions. Unlike truth tables that evaluate static statements, inference rules guide step-by-step reasoning, making them suitable for algorithms and automated proof strategies.

Truth Table Example (case: $P \rightarrow Q$)

P	Q	$P \rightarrow Q$
T	T	T

T	F	F
F	T	T
F	F	T

P is T and $P \rightarrow Q$ is T, forcing Q to be T.

2.7.4 Resolution Method

The Resolution Method is a single-rule, powerful inference technique used in propositional logic for automated theorem proving. It works by converting all statements into **Conjunctive Normal Form (CNF)** and applying the resolution rule to derive contradictions. The goal is to show that a negated statement leads to an inconsistency, proving the original statement true.

In CNF, every statement is a conjunction of disjunctions. Resolution works on pairs of clauses containing complementary literals. If one clause contains P and another contains $\neg P$, resolution eliminates them and combines the remaining parts. Repeated application of this rule eventually produces the **empty clause ($\perp$)**, which represents a contradiction.

Resolution is complete, meaning it can prove any statement that is logically valid. Its simple rule structure makes it efficient and suitable for computer implementation. It forms the basis of logic engines, SAT solvers, and parts of automated reasoning in AI.

Knowledge Representation and Reasoning (KRR)

Knowledge Representation and Reasoning (KRR) is a fundamental area of Artificial Intelligence (AI) concerned with how knowledge can be represented symbolically and how automated systems can reason about it. The goal of KRR is to enable machines to behave intelligently by storing, organizing, and manipulating information in a form that computers can process to draw meaningful conclusions.

In simple terms, KRR allows AI systems to “think” — by using stored information (knowledge) and applying logical rules (reasoning) to solve problems, make predictions, or understand situations. The efficiency and intelligence of an AI system largely depend on how well the knowledge is represented and how effectively the reasoning process is carried out.

2.6.1 Types of Knowledge

In Artificial Intelligence, knowledge is not uniform; it can be classified into several types based on its nature and use.